Heimalarm mit ESP32

Ein ESP32 liest die Ladeschale direkt aus – unter einer Sekunde, ohne Cloud. Zentrale ist eine UGREEN NAS oder ein Raspberry Pi.

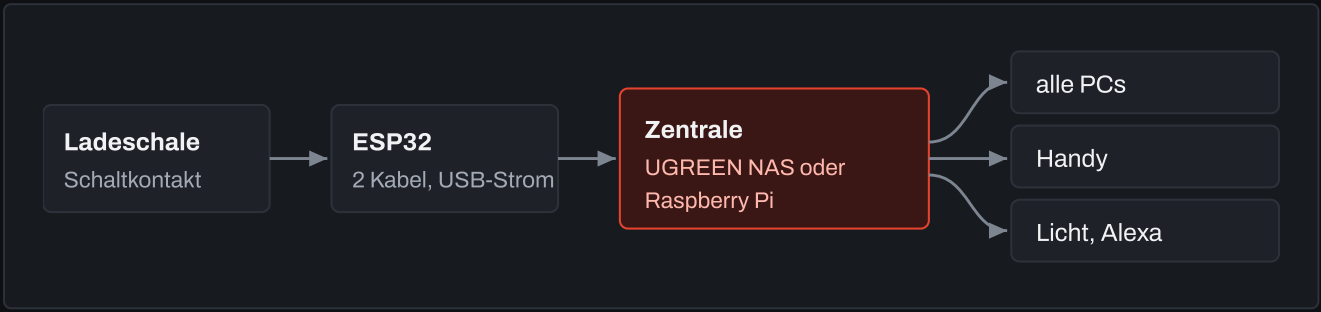
Dauer
etwa 90 Minuten

Für
UGREEN NAS (UGOS Pro) oder Raspberry Pi 4/5

Stand
Heimalarm 0.1.6 · Oktober 2026

Bevor du anfängst

Der ESP32 hängt am Schaltkontakt der Ladeschale. Schaltet der Melder bei Alarm, meldet der ESP32 das per WLAN an die Zentrale – und die gibt es an alle PCs und Handys weiter.



Einkaufsliste

- [ ] **ESP32-S3 DevKitC-1** (N16R8) mit **verlöteten** Stiftleisten, ca. 10 €
  Alternativ ESP32 DevKit oder Wemos D1 mini
- [ ] 2 **Jumper-Kabel** Buchse–Buchse
- [ ] **USB-C-Datenkabel** (nicht nur Ladekabel)
- [ ] Altes **USB-Netzteil** für Dauerstrom
- [ ] **Multimeter** zum Prüfen der Ladeschale

Außerdem

- [ ] Zentrale: **UGREEN NAS** *oder* **Raspberry Pi 4/5** mit Netzteil und microSD-Karte (32 GB)
- [ ] Windows-PC mit **Chrome** oder **Edge**
- [ ] Android-Handy
- [ ] Zugang zum Router, WLAN-Name und -Passwort (**2,4 GHz**)
- [ ] Ein langes eigenes **Passwort** für die Zentrale

Schritt	Inhalt	Dauer
1	Ladeschale prüfen und ESP32 anschließen	15 min
2	Zentrale einrichten – **A** UGREEN NAS *oder* **B** Raspberry Pi	20–30 min
3–4	Firmware vorbereiten, bauen und auf den ESP32 spielen	25 min
5–7	Heimalarm am PC und Handy, Testen	20 min

Schritt 1 · Ladeschale und ESP32

Dauer **15 min**

⚠ **Erst messen, dann anschließen.** Der ESP32 verträgt keine fremde Spannung am Eingang. Bei Spannung am Kontakt: nicht anschließen, sondern einen Optokoppler dazwischen setzen.

1. **Kontakt finden:** Hinten an der Ladeschale (z. B. Motorola TPG2200) sitzen zwei Kontaktstifte für ein externes Gerät.
2. **Spannung messen:** Multimeter auf Gleichspannung. Zwischen den Stiften muss **0 V** anliegen – im Ruhezustand und bei Alarm.
3. **Durchgang messen:** Multimeter auf Durchgang. In Ruhe **kein** Durchgang, bei Probealarm **Durchgang** (Piepton).
4. **Verbinden:** je ein Jumper-Kabel von einem Stift an den Eingang des ESP32 und vom anderen an **GND**. Die Reihenfolge ist egal.

Board	Eingang	Masse	Datei
ESP32-S3 DevKitC-1	`GPIO4`	`GND`	`alarm-ich-s3.yaml`
ESP32 DevKit (klassisch)	`GPIO27`	`GND`	`alarm-ich.yaml`
Wemos D1 mini (ESP8266)	`D5 (GPIO14)`	`G`	`alarm-ich-d1mini.yaml`

Kein Widerstand nötig – der ESP32 nutzt seinen eingebauten Pull-up. Die Datei aus der rechten Spalte brauchst du in Schritt 3.

✓ **Geklappt**, wenn beide Messungen wie oben aussehen und die Kabel fest sitzen. Strom bekommt der ESP32 erst nach dem Flashen.

Schritt 2A · Zentrale auf der UGREEN NAS

Dauer **20 min**

Hast du einen Raspberry Pi? Dann diese Seite überspringen und mit 2B weitermachen.

1. **Feste IP:** FRITZ!Box → **Heimnetz** → **Netzwerk** → bei der NAS auf den Stift → „Immer die gleiche IPv4-Adresse“ → **Übernehmen**. IP notieren.
2. **Docker** im App Center installieren. In **Dateien** → **docker** den Ordner `heimalarm` anlegen.
3. **Dateien:** GitHub `DanieldamHH112/Heimalarm` → **Code** → **Download ZIP**. In `docker/heimalarm` hochladen: den **Inhalt** von `zentrale` und zusätzlich den Ordner `esp32`.
4. **ESPHome einschalten:** `docker-compose.yml` mit **TextEdit** öffnen, beim Abschnitt `esphome:` die Zeile `profiles: ["esphome"]` löschen.
5. **Passwort:** am PC `.env.example` kopieren als `env.txt`, `MQTT_PASSWORD=` ausfüllen, hochladen, auf der NAS umbenennen in `.env`. Sie verschwindet danach aus der Ansicht – das ist richtig.
6. **Starten:** **Docker** → **Projekt** → **Erstellen**, Name `heimalarm`, Pfad `docker/heimalarm`, „sofort ausführen“ → **Bereitstellen**.

Geklappt, wenn im Protokoll von `heimalarm-mosquitto-1` „Adding password for user heimalarm“ steht und `heimalarm-dienst` mit `u'heimalarm'` verbunden ist.

Ausführlich mit Bildern: Anleitung „Funk-Alarmschalter + UGREEN NAS“, Schritte 2–4.

Schritt 2B · Raspberry Pi vorbereiten

Dauer **15 min**

Hast du eine NAS eingerichtet? Dann 2B überspringen. Der Pi läuft danach ohne Bildschirm und Tastatur.

1. **Raspberry Pi Imager** am PC installieren (`raspberrypi.com/software`) und die microSD-Karte einstecken.

2. Gerät: dein Pi-Modell. Betriebssystem: **Raspberry Pi OS (other)** → **Raspberry Pi OS Lite (64-bit)**. Speicher: die microSD-Karte.

3. Bei „Einstellungen anpassen?“ **Einstellungen bearbeiten**:

Hostname	`heimalarm`
Benutzer und Passwort	frei wählen und notieren
WLAN	nur, wenn kein LAN-Kabel möglich ist (Kabel ist zuverlässiger)
Dienste	**SSH aktivieren**, mit Passwort

4. **Schreiben**, warten, Karte in den Pi, Netzwerkkabel und Strom anschließen. 2 Minuten warten.

5. **Feste IP** im Router für das Gerät `heimalarm` einstellen (wie bei 2A) und notieren.

6. **Verbinden:** am PC **Windows-Taste** → **Terminal** öffnen und eingeben:

```
ssh benutzer@heimalarm.local
```

Beim ersten Mal `yes` tippen, dann das Pi-Passwort. Klappt `heimalarm.local` nicht, die IP verwenden.

Geklappt, wenn im Terminal `benutzer@heimalarm:~ $` steht.

Schritt 2B · Zentrale auf dem Raspberry Pi

Dauer **15 min**

Alle Befehle im Terminal aus dem letzten Schritt. Zeile für Zeile einfügen (Rechtsklick) und `Enter` drücken.

1 **Docker installieren** (2–5 min):

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
exit
```

Danach mit `ssh benutzer@heimalarm.local` neu verbinden.

2 **Heimalarm holen:**

```
git clone https://github.com/DanieldamHH112/Heimalarm.git
cd Heimalarm/zentrale
cp -r ../esp32 ./esp32
```

3 **Passwort festlegen:**

```
cp .env.example .env
nano .env
```

Bei `MQTT_PASSWORD=` dein Passwort eintragen. Speichern: `Strg` + `O`, `Enter`, Beenden: `Strg` + `X`.

4 **Starten:**

```
docker compose up -d
docker compose logs mosquitto
```

✓ **Geklappt**, wenn das Protokoll „Adding password for user heimalarm“ und „heimalarm-dienst … u'heimalarm'“ zeigt. Die Zentrale startet ab jetzt nach jedem Stromausfall von selbst.

⚠ **„MQTT_PASSWORD fehlt“?** `nano .env` erneut öffnen, Passwort prüfen, dann `docker compose up -d`.

Schritt 3 · Firmware vorbereiten

Dauer **10 min**

Die Firmware ist das Programm auf dem ESP32. Du trägst WLAN, Zentrale und deine Kennung ein – gebaut wird sie danach auf der Zentrale.

1 **ESPHome starten:**

NAS: läuft schon (Schritt 2A, Zeile 4). **Pi:** im Terminal `docker compose --profile esphome up -d esphome`

2 Im Browser `http://<IP der Zentrale>:6052` öffnen. Anmelden mit `heimalarm` und deinem `MQTT_PASSWORD`.

3 Oben rechts **Secrets** öffnen und ausfüllen:

```
wifi_ssid: "Name deines WLANs"        # 2,4 GHz
wifi_pw: "WLAN-Passwort"
ota_pw: "eigenes Update-Passwort"
mqtt_host: "IP der Zentrale"
mqtt_user: "heimalarm"
mqtt_pw: "dein MQTT_PASSWORD"
```

Save. Die Vorlage dafür ist `secrets.example.yaml`.

4 Bei der Karte für **dein Board** (Tabelle Schritt 1) auf **Edit**. Ganz oben steht `kennung: ich`. Für dich passt `ich` – bei weiteren Personen eine eigene Kennung wählen, z. B. `kollege`. **Save**.

✓ **Geklappt**, wenn **Validate** im Menü der Karte (⋮) keinen Fehler zeigt.

Schritt 4 · Firmware bauen und aufspielen

Dauer **15 min**

1. **Bauen:** auf der Karte `⋮` → **Install** → **Manual download**. Das dauert beim ersten Mal 5–15 Minuten (der Pi ist langsamer als die NAS).

2. **Herunterladen:** wenn fertig, **Factory format** wählen. Eine `.bin`-Datei landet im Download-Ordner.

   Die Menünamen können je nach ESPHome-Version leicht abweichen.

3. **ESP32 anstecken:** per USB-**Daten**kabel an den PC. Beim S3 die Buchse mit der Aufschrift **COM** bzw. **UART** nehmen.

4. **Aufspielen:** in Chrome oder Edge `web.esphome.io` → **Connect** → den USB-Anschluss wählen → **Install** → die `.bin`-Datei wählen → **Install**.

5. **ESPHome wieder stoppen:** wird nur zum Bauen gebraucht.

   NAS: **Container** → `heimalarm-esphome-1` → Stopp. **Pi:** `docker compose stop esphome`

⚠ **Kein Anschluss zur Auswahl?** Anderes Kabel probieren (viele sind nur Ladekabel). Beim S3: Taste **BOOT** gedrückt halten, einstecken, loslassen.

ESP32 in Betrieb nehmen

6. ESP32 vom PC trennen und an das **USB-Netzteil** neben der Ladeschale stecken. Die Kabel zur Ladeschale bleiben dran.

✓ **Geklappt**, wenn in der App (Schritt 5) die Übersicht **Mein ESP32: Online** zeigt – mit WLAN-Stärke und IP.

Schritt 5 · Heimalarm am PC

Dauer **5 min pro PC**

Auf **jedem** PC und Laptop, der bei Alarm reagieren soll. Beliebig viele.

1. **Programm laden:** `github.com/DanieldamHH112/Heimalarm/releases` → neueste Version → `Heimalarm_…_x64-setup.exe`.

2. **Installieren:** Datei öffnen. Zeigt Windows „Der Computer wurde durch Windows geschützt“: **Weitere Informationen** → **Trotzdem ausführen**.
   Das kommt, weil das Programm (noch) nicht kostenpflichtig signiert ist.

3. **Firewall:** Fragt Windows nach dem Netzwerkzugriff, **Private Netzwerke** erlauben.

4. **Betrieb:** **Einstellungen** → **Betrieb** → **Mit Zentrale**.

5. **Verbindung:** **Einstellungen** → **Verbindung**:

Adresse	IP der Zentrale aus Schritt 2, z. B. `192.168.178.43`
Port	`9001`
Benutzer	`heimalarm`
Passwort	dein `MQTT_PASSWORD`

   → **Speichern & verbinden**

6. **Alarmquelle:** **Einstellungen** → **Alarmquellen** → **Melder** einschalten.

7. **Kennung:** **Einstellungen** → **Alarm** → **Meine Kennung** = `ich` – genau wie im ESP32 (`kennung`).

8. **Was der PC tun soll:** **Einstellungen** → **PC bei Alarm**. Standard: Abfrage „Fährst du?“ mit 30 s Countdown, danach Ruhezustand.

✓ **Geklappt**, wenn die Übersicht **Zentrale: Verbunden** zeigt und unten links ein grünes **Verbunden** steht.

⚠ **Gemeinsamer PC?** Soll ein PC bei mehreren Personen ausgehen: **PC bei Alarm** → „Abfrage auch bei Alarm von“ die weiteren Kennungen eintragen.

Schritt 6 · Heimalarm am Handy

Dauer **5 min**

Android-Handy. Es meldet Alarme, gibt deine Antwort weiter und kann zusätzlich SYBOS, aPager oder Divera mitlesen.

1. **App laden:** Am Handy `github.com/DanieldamHH112/Heimalarm/releases` → `Heimalarm-v….apk`.
2. **Installieren:** Datei öffnen. Fragt Android nach „Unbekannte Apps installieren“: für den Browser **Zulassen**. Warnt Play Protect: **Trotzdem installieren**.
3. **Akku:** Beim ersten Start fragt die App nach der Ausnahme von der Akku-Optimierung → **Zulassen**. **Wichtig** – sonst beendet Android die App im Hintergrund.
4. **Betrieb und Verbindung:** **Mehr** → **Betrieb** → **Mit Zentrale**. Dann **Verbindung** mit denselben Daten wie am PC → **Speichern & verbinden**.
5. **Alarmquellen: Melder** an. Nutzt du SYBOS, aPager PRO oder Divera: die App dort ebenfalls einschalten.
6. **Mitlesen erlauben** (nur wenn eine Alarm-App an ist): bei der Alarm-App **Zugriff erlauben** → in der Android-Liste **Heimalarm** einschalten.
7. **Kennung:** **Mehr** → **Alarm** → **Meine Kennung** = `ich` – wie im ESP32 und am PC.

Geklappt, wenn am PC unter **Verbundene Apps** jetzt auch das Handy steht.

Nach einem **Neustart des Handys** die App einmal öffnen, damit der Hintergrund-Dienst wieder läuft.

Optional

- **Rückmeldung an Divera oder aPager:** bei der jeweiligen Alarmquelle einschalten (Standard aus).
- **Einsatz-Logbuch und Statistik:** **Einstellungen** → **Funktionen**.
- **App-Sperre** mit PIN und Fingerabdruck: **Einstellungen** → **Sicherheit**.

Schritt 7 · Testen

Dauer **10 min**

Von harmlos bis echt. Erst wenn alle vier klappen, ist Heimalarm einsatzbereit.

1. **Testalarm** – **Einstellungen** → **Test & Diagnose** → **Testalarm**.
   Der PC zeigt „Fährst du?“, es passiert aber nichts. Gut zum Ausprobieren.
2. **Verbindungstest** – gleiche Seite → **Jetzt testen**.
   Zentrale und Zentralen-Dienst müssen grün sein.
3. **Kontakt simulieren** – wie ein echter Melder-Alarm, geht an alle Geräte. Routinen laufen nur im Testmodus.
4. **Echter Probealarm** – Melder in die Ladeschale, Probealarm auslösen lassen. Die Meldung kommt in unter einer Sekunde.

Alles bereit, wenn beim Probealarm **jeder PC** „Fährst du?“ zeigt und das **Handy** eine Meldung bringt.

Wenn etwas nicht reagiert

Was passiert	Erst prüfen
Kein Gerät reagiert	Ist unter **Alarmquellen** der **Melder** an? Steht überall dieselbe **Kennung**?
Nur ein PC reagiert nicht	Ist er verbunden? Läuft eine **Ruhezeit** (**PC bei Alarm**)?
Handy meldet im Hintergrund nichts	Akku-Ausnahme erlaubt? **Einstellungen** → **Android**.
„Mein ESP32: Offline“	Strom? WLAN 2,4 GHz? `mqtt_host` und `mqtt_pw` in den Secrets richtig?
ESP32 online, Alarm kommt nicht	Kabel an GPIO-Eingang und GND? Kennung im ESP32 und in der App gleich?

Pflege und Fehler

Updates

- **Apps:** PC und Handy melden neue Versionen beim Start → **Installieren**.
- **Zentrale auf dem Pi:**
  `cd Heimalarm && git pull && cd zentrale && docker compose up -d --build`
- **Zentrale auf der NAS:** neue Dateien hochladen (nicht `.env` und die `data`-Ordner), Container und Images von `dienst` löschen, **Neu bereitstellen**.
- **ESP32:** nur nötig, wenn es eine neue Firmware gibt – dann ESPHome starten und **Install** → **Wirelessly**. Ohne Kabel.

Häufige Fehler

Problem	Lösung
ESPHome-Seite (Port 6052) lädt nicht	Container `esphome` gestartet? Auf der NAS die Zeile `profiles: ["esphome"]` gelöscht?
Bauen bricht ab	Fehlermeldung lesen – meist fehlt ein Eintrag in den Secrets.
web.esphome.io findet den ESP32 nicht	Datenkabel, andere USB-Buchse, BOOT-Taste beim Einstecken halten.
ESP32 verbindet sich nicht mit dem WLAN	Nur 2,4 GHz. WLAN-Name genau wie im Router (Groß-/Kleinschreibung).
Fehlalarme ohne Einsatz	In der App **Einstellungen** → **ESP32** die **Mindestdauer** erhöhen.
App: Zentrale „Getrennt“	IP, Port `9001`, Benutzer `heimalarm`, Passwort prüfen.
NAS: „Compose-Datei verloren“	Projekt löschen (ohne Dateien) und neu erstellen.

⚠ **Nie löschen:** `.env`, `mosquitto/data`, `esp32/secrets.yaml`. Den Pi nicht einfach vom Strom ziehen – vorher `sudo shutdown now`.